

Name: Dayanand Kori

DESIGN	1
API Contract	1
Data Model	4
Processing Logic	8
Walk through these failure scenarios	12
A) Concurrent duplicate:	12
b) Missing or late version:	12
c) Crash after saving:	13
d) Worker interruption:	13
e) Newer version during processing:	14
F) Results out of order:	14
G) AI outage and retry exhaustion:	15
h) Inconsistent identity fields:	16
Show how you would verify it	16
Test Cases	16
Investigating a stuck or late summary	17
LIMITATIONS	18

ASSUMPTION

1. Code is Postgres SQL plus language-neutral pseudocode
2. The partner retries failed deliveries using the same *event_id* (as mentioned in the brief). Since the retry policy is unspecified, the design does not depend on a particular retry pattern; duplicate and concurrent deliveries are handled idempotently, so retries remain safe.
3. *generate_summary* is an external call with no cancellation API. Once a call is in flight, its cost is already incurred.
4. The worker times out *generate_summary* after 30 seconds. This only stops the worker from waiting; the provider may still complete and bill the call.

5. One shared AI dependency. All workers call the same *generate_summary* provider under a shared quota, so an outage or rate limit affects them all at once.
6. Authentication and authorization are out of scope.

DESIGN

API Contract

❖ POST

Outcome	Code	Rationale
Accepted / current	201 Created	A job resource was created.
Duplicate	200 OK	The caller's goal was "make sure you have this event". The service has it. Returning an error would make correct retry behaviour look like failure in the partner's dashboards.
Stale	200 OK	A valid late event may be acknowledged successfully even if it does not change current state; there is nothing for the caller to fix.
Identity mismatch	409 Conflict	The conflict is permanent and caller-side, so retrying will not help. <i>409 Conflict</i> is appropriate because the request conflicts with existing state; <i>422</i> would imply the request is invalid on its own.
Payload too large	413 Payload Too Large	The body is over the size limit. It's rejected before parsing. Resending the same body can't succeed.
Payload conflict	409 Conflict	The (encounter_id, version) already has a different payload recorded. This is a source defect: the stored version is kept and retrying will not succeed.
Malformed request	400 Bad Request	A required field is missing, version is not a positive integer, or encounter_type is not one of the three allowed values. Nothing is stored.

Every response to a well-formed event carries `encounter_id` and `current_version`, including conflicts.

❖ GET

All three job states return 200 OK: the resource exists and its state is being reported. A failed job is a successful read of a failed thing.

1. **Processing** — status, version being worked on, *accepted_at*, computed *sla_breached*. No summary.
2. **Ready** — status, summary text, and the version it describes (always equal to *current_version*), completion timestamp, *sla_breached*.
3. **Failed** — status, attempt count, error classification (*ai_timeout*, *rate_limited*, etc). Never the raw provider error, which could echo transcript content.
4. **Unknown encounter** — 404 Not Found.

Example Requests and Responses

```
POST /encounters/events
Content-Type: application/json
{ "event_id": "evt-102", "encounter_id": "enc-42", "patient_id":
"pat-77",
  "encounter_type": "TelephoneTriage", "version": 12,
  "payload": { "transcription": "Nurse: Thanks for
calling, ..." } }
```

```
Accepted / current
-> 201 Created
Location: /encounters/enc-42/summary
{ "outcome": "accepted", "encounter_id": "enc-42", "version":
12,
  "current_version": 12, "job_status": "queued" }
```

```
Duplicate (retry of evt-102, or another event_id for enc-42 v12)
-> 200 OK
{ "outcome": "duplicate", "encounter_id": "enc-42", "version":
12,
  "current_version": 13 }
```

```
Stale (v11 arrives after v12 is stored)
-> 200 OK
{ "outcome": "stale", "encounter_id": "enc-42", "version": 11,
  "current_version": 12 }
```

```
Identity mismatch (enc-42 is bound to a different patient)
-> 409 Conflict
{ "outcome": "identity_conflict", "encounter_id": "enc-42",
  "current_version": 12, "conflicting_field": "patient_id",
```

```
    "message": "Contradicts stored encounter identity. Retrying  
will not succeed." }
```

Payload conflict (v12 already stored with a different
transcript)

-> 409 Conflict

```
{ "outcome": "payload_conflict", "encounter_id": "enc-42",  
  "version": 12,  
    "current_version": 13,  
    "message": "A different payload is already recorded for this  
version." }
```

GET /encounters/enc-42/summary

Processing (queued jobs are also reported as processing)

-> 200 OK

```
{ "encounter_id": "enc-42", "patient_id": "pat-77",  
  "encounter_type": "TelephoneTriage", "current_version": 12,  
  "status": "processing", "summary": null,  
  "accepted_at": "2026-09-18T10:04:11Z", "sla_breached": false }
```

Ready

-> 200 OK

```
{ "encounter_id": "enc-42", "patient_id": "pat-77",  
  "encounter_type": "TelephoneTriage", "current_version": 12,  
  "status": "ready", "summary_version": 12,  
  "summary": "Patient reported sore throat and fever ...",  
  "accepted_at": "2026-09-18T10:04:11Z",  
  "completed_at": "2026-09-18T10:04:17Z", "sla_breached":  
false }
```

Failed

-> 200 OK

```
{ "encounter_id": "enc-42", "patient_id": "pat-77",  
  "encounter_type": "TelephoneTriage", "current_version": 12,  
  "status": "failed", "summary": null, "attempts": 5,  
  "error_class": "ai_unavailable",  
  "accepted_at": "2026-09-18T10:04:11Z",  
  "completed_at": "2026-09-18T10:22:48Z", "sla_breached": true }
```

Unknown encounter

-> 404 Not Found

```
{ "error": "encounter_not_found", "encounter_id": "enc-999" }
```

summary_version always equals **current_version** by construction; it is returned so clients
can assert that rather than trust it.

Data Model

Table	Cardinality
<i>encounters</i>	1 per encounter
<i>encounter_events</i>	Many per encounter
<i>summary_jobs</i>	1 per accepted version
<i>job_attempts</i>	Many per job

Initially, the model had only three tables, but `summary_jobs.attempts` was being used for three different purposes: fencing, retry budgeting, and tracking paid AI calls. `job_attempts` separates these concerns and provides a durable record for each individual attempt.

How identity relates to encounters and summaries. `patient_id` and `encounter_type` are stored once, on `encounters`, keyed by `encounter_id`, and written only when the encounter is first accepted. Events and jobs do not repeat them; a summary reaches its patient through `summary_jobs.encounter_id → encounters`. Because these fields can never change (scenario H), every summary for an encounter belongs to the same patient and type.

Schema

```
CREATE TYPE encounter_type AS ENUM ('Appointment',
'MedicationRefill', 'TelephoneTriage');
CREATE TYPE job_status AS ENUM ('queued', 'processing',
'ready', 'failed', 'superseded');

-- One row per encounter. The only mutable table.
CREATE TABLE encounters (
    encounter_id TEXT PRIMARY KEY,
    patient_id TEXT NOT NULL, -- fixed for
life of encounter
    encounter_type encounter_type NOT NULL, -- fixed for
life of encounter
    current_version INTEGER NOT NULL DEFAULT 0 CHECK
(current_version >= 0),
-- 0 only inside
an ingestion txn (shell row); never committed
    transcription TEXT, -- snapshot at
current_version; NULL only while version is 0
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Append-only. Exists to make duplicates a constraint
violation, not a race.
```

```

CREATE TABLE encounter_events (
    event_id          TEXT PRIMARY KEY,          -- catches plain
retries
    encounter_id      TEXT      NOT NULL REFERENCES
encounters(encounter_id),
    version           INTEGER NOT NULL CHECK (version > 0),
    received_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT uq_encounter_version UNIQUE (encounter_id,
version) -- catches source defects
);

```

-- One row per accepted version. Input frozen at creation.

```

CREATE TABLE summary_jobs (
    job_id            BIGSERIAL PRIMARY KEY,
    encounter_id       TEXT      NOT NULL REFERENCES
encounters(encounter_id),
    version           INTEGER NOT NULL,
    event_id          TEXT      NOT NULL REFERENCES
encounter_events(event_id),
    input_transcription TEXT      NOT NULL,          -- immutable
snapshot; worker never reads encounters
    status            job_status NOT NULL DEFAULT 'queued',
    attempts          INTEGER     NOT NULL DEFAULT 0,  --
fencing token; only ever increases
    redrive_generation INTEGER     NOT NULL DEFAULT 0,  --
bumped on redrive; scopes the retry budget
    summary           TEXT,          -- written once
on success
    superseded_by_version INTEGER,          --
current_version at the moment this job was marked superseded
    accepted_at       TIMESTAMPTZ NOT NULL DEFAULT now(), --
SLA clock starts here
    started_at        TIMESTAMPTZ,
    completed_at       TIMESTAMPTZ,
    next_attempt_at    TIMESTAMPTZ NOT NULL DEFAULT now(), --
backoff scheduling
    lease_expires_at   TIMESTAMPTZ,          -- reclaims jobs
from crashed workers
    CONSTRAINT uq_job_encounter_version UNIQUE (encounter_id,
version)
);

```

-- One partial index per branch of the claim query's OR (section 5)

```

CREATE INDEX idx_jobs_due ON summary_jobs (next_attempt_at)
    WHERE status = 'queued';          -- queued and due

```

```

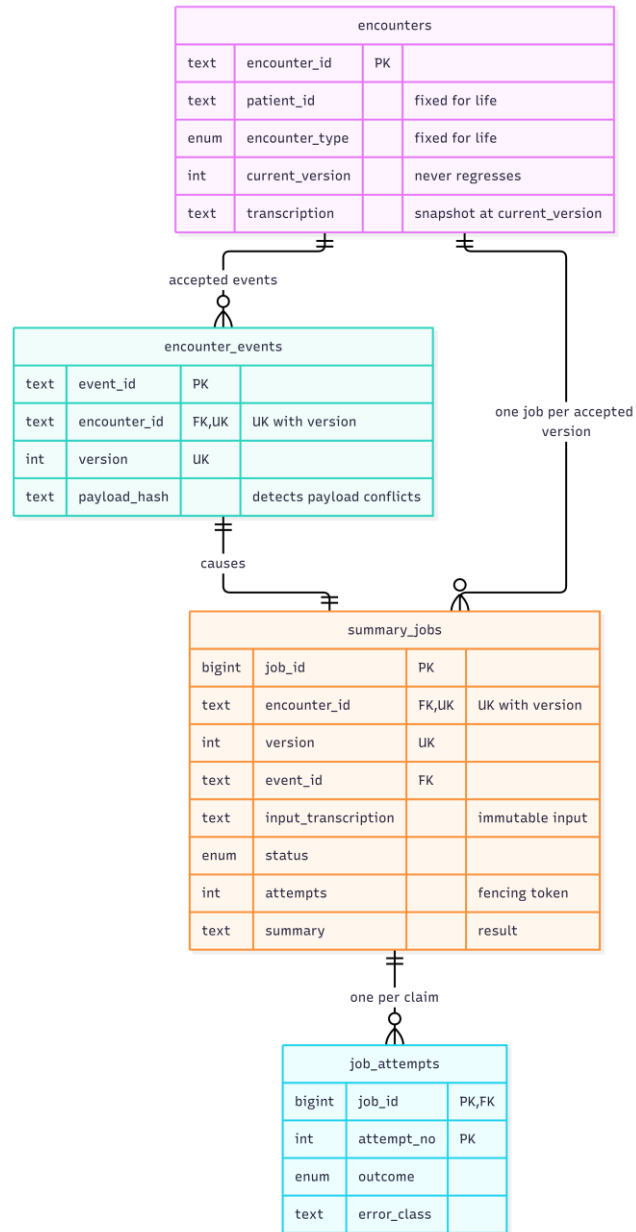
CREATE INDEX idx_jobs_expired_lease ON summary_jobs
(lease_expires_at)
    WHERE status = 'processing';           -- processing
with an expired lease

CREATE TYPE attempt_outcome AS ENUM (
    'in_flight',          -- claimed, not yet finished
    'succeeded',          -- result written as ready
    'superseded',         -- result written, but a newer version
is current
    'skipped',            -- pre-call check found the job
obsolete; no AI call made
    'transient_error',    -- timeout / rate limit / unavailable;
job requeued with backoff
    'lease_expired',      -- owner presumed dead; job reclaimed by
another worker
    'discarded'           -- call returned after losing ownership;
result thrown away
);

-- One row per claim. The per-attempt audit and cost record.
CREATE TABLE job_attempts (
    job_id                BIGINT    NOT NULL REFERENCES
summary_jobs(job_id),
    attempt_no            INTEGER NOT NULL,          -- equals
summary_jobs.attempts at claim time
    redrive_generation    INTEGER NOT NULL,          -- copied from
the job at claim time
    worker_id             TEXT      NOT NULL,
    outcome               attempt_outcome NOT NULL DEFAULT
'in_flight',
    error_class           TEXT,                      -- fixed enum
value only, never patient content
    started_at            TIMESTAMPTZ NOT NULL DEFAULT now(),
    finished_at           TIMESTAMPTZ,
    PRIMARY KEY (job_id, attempt_no)
);

```

ERD



Processing Logic

Ingestion Psuedocode:

```
function ingest(request):
    if request.body_size > 1 MB:
        return 413
    before parsing; no DB work
```

-- rejected

```

BEGIN                                                    -- one
transaction for everything below
    INSERT encounter_shell (version 0) ON CONFLICT DO NOTHING
    enc = SELECT identity, current_version FOR UPDATE    --
serialises events per encounter

    if enc.patient_id != event.patient_id or
enc.encounter_type != event.encounter_type:
        ROLLBACK; log + metric
        return 409 identity_conflict

    hash = sha256(event.payload.transcription)
    inserted = INSERT encounter_events (... , hash) ON CONFLICT
DO NOTHING
    if not inserted:                                     --
event_id or (encounter_id, version) seen
        existing_hash = SELECT payload_hash of the matching
event
        ROLLBACK
        if existing_hash == hash: return 200 duplicate
(enc.current_version)
        else:                                           return 409 payload_conflict

    updated = UPDATE encounters SET version, transcription
        WHERE current_version < event.version    --
compare-and-swap
    if not updated:
        ROLLBACK
        return 200 stale (enc.current_version)

    INSERT summary_jobs (... , input_transcription = event
transcription)
    COMMIT                                                    -- the
commit is the hand-off to the worker
    return 201 accepted

```

The AI call is intentionally kept outside any transaction, because a crash during the call would erase the record of the spend, and the transaction would hold database resources until the call finished.

Step	Transcation
Claim	txn 1
Pre-call check	txn 2
generate_summary	none
Result or failure	txn 3

Worker Psuedocode:

```
loop forever:
    job = claim()          -- txn 1: breaker check, SKIP LOCKED,
    attempts+1 (fencing token),
    --                      60s lease, close any
    abandoned attempt as lease_expired,
    --                      reclaim budget check, open
    attempt row
    if job is None:        -- nothing due, or breaker not closed
        try_probe_if_cooldown_passed()
        sleep briefly; continue
    if job.failed_on_reclaim:    -- budget exhausted on
    reclaim; txn 1 marked it failed
        continue

    if precall_supersede(job):    -- txn 2: fenced
    conditional update
        continue                -- job superseded, attempt
    skipped, no call, no cost

    try:
        summary = generate_summary(job.input_transcription,
        timeout = 30s)    -- no txn open
    except TransientError as e:    -- timeout, rate limit,
    unavailable
        record_failure(job, e.error_class)    -- txn 3a, fenced:
    queued + backoff,
        -- or failed at 5
    attempts; breaker trip check
        continue

    if not guarded_write(job, summary):    -- txn 3b, fenced:
    ready or superseded
        mark_attempt_discarded(job)    -- lost ownership;
    paid call, result unused
```

Claiming a job

Workers take the oldest due job using FOR UPDATE SKIP LOCKED, so two workers never grab the same job and never wait on each other. Claiming does three things: it marks the job processing, increases attempts (which acts as the worker's ownership token), and sets a 60-second lease. If a worker dies, its lease runs out and another worker can reclaim the job. Before claiming, the worker checks the circuit breaker, and it claims nothing while the AI service is down.

Pre-call check

Right after claiming, the worker checks whether its job's version is still the encounter's current version. If a newer version has arrived, the job is marked **superseded** and no AI call is made. This is the only point where money can actually be saved on obsolete work.

Writing the result

The worker saves its result only if it still owns the job, meaning its **attempts** value hasn't changed. If it does, the job becomes **ready** when its version is still current, or **superseded** when a newer one has arrived. A worker that lost its job while waiting on a slow call has its result rejected. Because the saving commit is itself the acknowledgement, a crash after saving loses nothing, and a crash before saving just leads to a retry.

Stale-result protection

There are two layers. GET always looks up the summary for the encounter's *current* version, so an older result can never be found. And the result write labels obsolete results **superseded**, so the job status always tells the truth.

Retries and cost control

Three mechanisms keep spend bounded:

- **Retry budget:** each job gets at most 5 attempts. Every attempt that may have been paid for counts toward it, including crashes and timeouts. Skipped attempts don't count.
- **Backoff:** after a failure, the job waits before retrying (about 10, 20, 40, then 80 seconds, with some randomness), so retries don't hammer a struggling service.
- **Circuit breaker:** if 11 of the last 20 calls fail, ignoring calls older than 10 minutes, workers stop calling the AI entirely. One test call is sent every 30 seconds, using a fake transcript, until the service recovers. This caps spend during an outage no matter how many jobs are waiting. At very low traffic the breaker may not trip at all, which is acceptable: with few calls happening, per-job budgets already bound spend.

Failed jobs and redrive

A job that uses all 5 attempts becomes **failed** and is never retried automatically. Automatic retries would mean unlimited spend during a long outage. An operator can redrive it, which gives it a fresh budget, but only if its version is still current. Otherwise it's marked **superseded**.

Obsolete work: the trade-off

The AI call can't be cancelled once sent, so obsolete work is only stopped *before* the call, by the pre-call check. Work already in flight is allowed to finish, its result is stored as **superseded**, and it's kept for cost tracking and debugging. Trying to cancel in-flight calls would add complexity without saving any money.

SLA

The target is a ready summary within 10 seconds of acceptance. A scheduled check counts late jobs and alerts if lateness persists. GET also shows `sla_breached` to the client. Being late never changes a job's status: a late job keeps processing, and only running out of retries makes it failed.

Walk through these failure scenarios

A) Concurrent duplicate:

Two service instances receive the same event concurrently.

What happens. Both instances run the ingestion transaction, and the database decides. A locks the encounter row first, stores the event and job, and commits. B's event insert then hits the `event_id` primary key and rolls back as a duplicate.

Stored state. One event row, one version update, one job.

Client sees. A returns 201 (accepted). B returns 200 (duplicate, `current_version`: 12).

Recovery. The single job is processed normally. If A crashes before committing, its transaction rolls back and B is accepted instead, and A's retry becomes a duplicate. Either way, the event is accepted exactly once.

b) Missing or late version:

Version 12 arrives while version 10 is stored.

Version 11 arrives later, or never arrives.

What happens. v12 is newer than v10, so it's accepted, even though v11 is missing. Each payload is a complete snapshot, so v12 doesn't depend on v11. If v11 arrives later, it's older than the stored v12, so it's classified as stale and rolled back. If v11 never arrives, nothing happens, because the design never waits for missing versions.

Stored state. The encounter is at v12, with v12's transcript and one job for v12. Nothing from v11 is stored.

Client sees. v12 gets 201 (accepted). v11 gets 200 (stale, current_version: 12). GET shows the v12 summary.

Recovery. None needed. A retry of v11 gets the same stale response every time.

c) Crash after saving:

The encounter update is saved, but the service crashes before handing work to the summary worker.

What happens. This state can't occur. The job is created in the same transaction as the encounter update, so both are saved or neither is. There's no separate hand-off step: workers find work by reading the jobs table.

- **Crash before commit:** everything rolls back, and nothing is saved.
- **Crash after commit, before the response:** the encounter and its job are both saved.

Stored state. Either nothing, or the full accepted state: the encounter at the new version, its event row, and a queued job.

Client sees. The original request gets no response. On retry, the partner gets 201 if the crash was before commit, or 200 duplicate if it was after.

Recovery. Automatic, through the partner's retry and a worker picking up the job. The summary is generated exactly once either way.

d) Worker interruption:

A worker crashes during processing, or after saving a result but before acknowledging the job.

Crash during processing. The job stays processing with nothing written. After its 60-second lease expires, another worker reclaims it, marks the dead worker's attempt lease_expired, and tries again. The crashed attempt counts toward the retry budget, since the call may already have been paid for. A worker that was only stalled, not dead, has its late result rejected by the fencing token.

- **Stored state:** job processing under a new worker, with the crashed attempt recorded.

- **Client sees:** `processing`, then `ready`. The summary is late, with `sla_breached: true`.
- **Recovery:** automatic, through lease expiry. A job that crashes the worker every time fails after 5 attempts.

Crash after saving, before acknowledging. There's no separate acknowledgement step: the commit that saves the result is the acknowledgement. Once saved, the job is `ready`, and workers only claim `queued` jobs or `processing` jobs with an expired lease, so it's never picked up again.

- **Stored state:** job `ready` with its summary.
- **Client sees:** `ready`. The crash is invisible to them.
- **Recovery:** none needed, and no second paid call is made.

e) Newer version during processing:

Version 12 is actively being summarized when version 13 arrives. What happens to version 12's work and result?

What happens. v13 is accepted, and a v13 job is created for any free worker. v12's AI call is already in flight and can't be cancelled, so it runs to completion. When v12's result is written, the version check sees that v13 is now current, so the v12 job is marked `superseded`, not `ready`. The summary is kept, with `superseded_by_version = 13`. If v13 had arrived before v12's call was sent, the pre-call check would have skipped the call entirely, at no cost.

Stored state. The encounter is at v13. The v12 job is `superseded`, with its summary kept. The v13 job is `processing`, then `ready`.

Client sees. Only v13: `processing`, then `ready`. GET always reads the current version, so v12's result is never shown.

Recovery. None needed. v12's call is a sunk cost, and it's kept for cost tracking and debugging.

F) Results out of order:

Version 13 finishes summarizing before version 12.

What happens. Each result is written only to its own job row, so the order of arrival doesn't matter. v13's result lands first; its version is current, so its job becomes `ready`. v12's result lands

later; its version is no longer current, so its job becomes **superseded**, with the summary kept and `superseded_by_version = 13`. v12's write can never touch v13's row.

Stored state. The encounter is at v13. The v13 job is **ready**, and the v12 job is **superseded**.

Client sees. Only v13: **processing**, then **ready** with v13's summary. v12's result is never shown, whether it finishes before or after v13.

Recovery. None needed. If the summary were instead stored on the encounter row, the last write would win, and v12 finishing late would overwrite the newer summary. Per-job results plus reads through `current_version` make that impossible.

G) AI outage and retry exhaustion:

The AI service is unavailable for 20 minutes.

Backoff and retry. A failed call fails the *attempt*, not the job. The job goes back to **queued** and waits about 10, 20, 40, then 80 seconds, with some randomness so retries don't all return at once. Once 11 of the last 20 calls have failed, the circuit breaker opens and workers stop calling the AI entirely. Every 30 seconds, one worker sends a test call with a fake transcript. When a test call succeeds, the breaker closes and work resumes, oldest first.

Cost. Every call that may have been paid for counts toward a job's budget of 5, including timeouts. While the breaker is open, no job's budget is used at all. The whole 20-minute outage costs roughly 20 to 40 test calls, however many jobs are waiting. Without the breaker, each queued job could spend up to 5 paid calls.

Retry exhaustion. In this scenario, almost no job reaches **failed**: each uses one or two attempts before the breaker opens, then none while it's open. Exhaustion happens when the breaker can't help, for example in a partial outage or with a transcript that always fails. The job becomes **failed** after 5 attempts and is never retried automatically, so spend stops there.

Inspect and redrive. Each failed job's attempt history shows every attempt's outcome, error type and timing, so provider failures can be told apart from worker crashes. An operator can redrive a failed job, which gives it a fresh budget of 5, but only if its version is still current. Otherwise it's marked **superseded**, so no money is spent on a summary nobody will see.

Client sees.

- **During the outage:** **processing**, with `sla_breached: true` after 10 seconds.
- **After recovery:** **ready**, still marked as breached.

- **If the job's budget ran out:** failed, with the attempt count and error type.

New POSTs are accepted normally throughout, because ingestion doesn't depend on the AI.

h) Inconsistent identity fields:

An event for an existing `encounter_id` arrives with a different `patient_id` or `encounter_type` than previously stored.

What happens. The first patient wins. Ingestion locks the encounter row and compares the stored `patient_id` and `encounter_type` with the incoming ones. On any mismatch, the transaction rolls back and the event is rejected. A structured log entry and a dedicated metric record the conflict. An `encounter_type` mismatch is treated exactly like a `patient_id` mismatch.

Stored state. Unchanged. The encounter stays bound to its original patient, and its versions, jobs and summaries are untouched.

Client sees. 409 Conflict with "outcome": "identity_conflict" and the conflicting field name, and the same 409 on every retry. The stored `patient_id` is never echoed back, so a caller with the wrong data can't learn the real patient. GET for the encounter is unaffected.

Recovery. Deliberately not automatic. The fix belongs in the partner's mapping, and the metric and log give operators what they need to raise it. If two first events for a new encounter race with different patients, the one that commits first wins, and the other gets 409.

Show how you would verify it

Test Cases

Test	Setup and Action	Expected outcome
Concurrent duplicate	Two instances receive the same evt - 102 at once	One 201, one 200 duplicate. One event row, one job.
Version never regresses	v12 and v13 arrive concurrently, repeated in both orders	Encounter ends at v13. v13 is ready; v12 is superseded or stale. No GET ever returns v12's

		summary, sampled throughout.
Identity race on a new encounter	First two events for enc -900 arrive at once with different patients	One 201, one 409 with no patient ID echoed. Only the winner's event and job are stored.
Crash around the ingestion commit	Kill the service before commit, then after commit; the partner retries	Before commit: nothing stored, and the retry gets 201. After commit: the retry gets 200 duplicate. Exactly one job either way.
AI outage	100 queued jobs; the AI fails for a simulated 20 minutes	Breaker opens after 11 of 20 failures. Paid job calls stay near 11 plus calls already in flight, not about 500. Probes use only the synthetic transcript. No job fails; all reach ready after recovery.

Investigating a stuck or late summary

Metrics:

- breaker state
- queue depth, split into due-now and waiting-on-backoff
- busy workers versus worker slots
- count of late jobs and age of the oldest unfinished job
- AI call latency and call count
- attempts by outcome and error type

Timestamps: `accepted_at`, `started_at` and `completed_at` on each job. These split lateness into queue wait (`started_at` - `accepted_at`) and processing time (`completed_at` - `started_at`). Each attempt also has its own start and finish times.

Alerts:

- the oldest unfinished job over 60 seconds for 2 minutes
- the breaker staying open
- any job reaching `failed`
- any identity conflict

Triage: first check whether one job or many are late. If many, look at the breaker, queue depth and worker saturation. If one, look at its timestamps, then its attempt history. A run of `lease_expired` points to a poison transcript, and a run of `transient_error` points to the AI.

No patient content: logs and metrics carry only IDs, statuses, fixed error categories, timestamps and durations. Transcripts, summaries and raw provider errors are never logged.

LIMITATIONS

No access control. This is the largest gap. Any caller can submit events and read any encounter's summary and patient linkage. A real deployment needs authenticated callers, access scoped to the patients each caller may see, and an audit trail of reads. It's left out because the brief specifies no auth model.

Summaries are not validated. `ready` means a summary was generated, not that it's correct, so a hallucinated medication or dose would be shown as current. A fix would check facts in the summary against the transcript before serving it, and flag failures for review rather than retrying them.

A wrong first binding can't be corrected through the API. "First patient wins" assumes the first event was right. If it wasn't, every later correct event is rejected with `409`. Fixing it needs a manual, audited operator action.

Non-retryable errors are not distinguished. Every AI error is treated as transient, so an input the service always rejects wastes up to 5 paid calls before failing. It could also briefly trip the breaker. Once the provider's real error codes are known, permanent errors should fail immediately.

Encounter-level starvation is not detected. If an encounter's versions arrive faster than summaries can be generated, every job is superseded before it finishes. The client never gets a summary, yet no job ever counts as late. Catching this needs a per-encounter measure: time since the last ready summary.

No retention or deletion policy. Every version keeps its own transcript copy, and superseded summaries are retained, so patient content grows without limit.

